

Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Deep Learning
Relazione Progetto

Image Enhancement with Deep
Learning

Studio e confronto di metodi di restauro di immagini
mediante reti neurali convoluzionali

Autori:

Giuseppe BELLAMACINA

Salvo IURATO

Mattia CAMPANELLA

Daniele BARBAGALLO

Professore del corso:

Prof. ...

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Obiettivo del progetto	3
1.2	Suddivisione del lavoro	3
1.3	Dataset	3
1.4	Pipeline generale	4
2	Degradazioni	5
2.1	Rumore Gaussiano	5
2.2	Quantizzazione e Dithering	5
2.2.1	Quantizzazione semplice	5
2.2.2	Random Dithering	5
2.2.3	Ordered Dithering (Bayer)	6
2.2.4	Floyd-Steinberg Dithering	6
2.3	Salt-and-Pepper	6
2.4	Generazione automatica del dataset	6
3	Infrastruttura comune	7
3.1	Dataset e Data Loading	7
3.1.1	Estrazione delle patch	7
3.1.2	Data augmentation	7
3.1.3	DataLoader	8
3.2	Funzioni di Loss	8
3.2.1	Combined Loss (L1 + SSIM)	8
3.2.2	Perceptual Loss (VGG)	8
3.2.3	Combined Perceptual Loss	8
3.3	Sistema di Esperimenti	8
3.3.1	Struttura delle directory	9
3.3.2	Ripresa e fork	9
3.4	Checkpointing	9
3.5	Notifiche Telegram	9
4	Architetture dei Modelli	10
4.1	UNet	10
4.1.1	Architettura	10
4.1.2	Configurazione	11
4.2	UNet Residual	11
4.2.1	Anti-checkerboard	11
4.3	Attention UNet	11
4.3.1	Attention Gate	11

4.3.2	Livelli di attenzione	12
4.3.3	Normalizzazione	12
4.3.4	Adaptive Blending	12
4.4	DnCNN	12
4.4.1	Architettura	12
4.5	Denoising Autoencoder	13
4.5.1	Architettura	13
4.6	RIDNet	13
4.6.1	Enhancement Attention Block (EAB)	13
4.6.2	Architettura completa	13
4.7	Pix2Pix	14
4.7.1	Generatore	14
4.7.2	PatchGAN Discriminator	14
4.7.3	Modalità di apprendimento	14
4.8	NAFNet	14
4.9	GAN	14
4.10	Riepilogo delle architetture	15
5	Training	16
5.1	Pipeline di addestramento	16
5.1.1	Mixed Precision (AMP)	16
5.1.2	Learning Rate Warmup	16
5.1.3	Gradient Clipping	16
5.1.4	Early Stopping	16
5.1.5	Gestione errori OOM	17
5.2	Training Pix2Pix	17
5.2.1	Training del Discriminatore	17
5.2.2	Training del Generatore	17
5.2.3	Diagnostica del rumore	17
5.3	Configurazioni degli esperimenti	17
5.3.1	Note sugli esperimenti	18
6	Valutazione	19
6.1	Metriche	19
6.1.1	PSNR (Peak Signal-to-Noise Ratio)	19
6.1.2	SSIM (Structural Similarity Index)	19
6.1.3	MAE e MSE	19
6.2	Inferenza su immagini full-resolution	19
6.2.1	Sliding Window	20
6.2.2	Hann Window Blending	20
6.3	Valutazione del dataset	20

Capitolo 1

Introduzione

1.1 Obiettivo del progetto

Il presente progetto si propone di studiare e confrontare diversi approcci basati su reti neurali convoluzionali (CNN) per il problema dell'*image enhancement*, ovvero il restauro di immagini corrotte da degradazioni controllate. L'obiettivo è duplice: da un lato implementare una pipeline completa e modulare che copra l'intero ciclo *degradazione* $\rightarrow$ *addestramento* $\rightarrow$ *valutazione*; dall'altro analizzare empiricamente le differenze architetturali tra i modelli proposti, confrontandone le prestazioni in termini di metriche quantitative (PSNR, SSIM) e qualità percettiva.

1.2 Suddivisione del lavoro

Il progetto è stato sviluppato da quattro componenti, ciascuno responsabile di specifiche architetture:

- **Giuseppe Bellamacina** — UNet, UNet Residual, Attention UNet e infrastruttura comune (degradazioni, pipeline di training, valutazione, utilità);
- **Salvo Iurato** — Denoising Autoencoder, DnCNN e RIDNet;
- **Mattia Campanella** — NAFNet;
- **Daniele Barbagallo** — Pix2Pix e GAN.

1.3 Dataset

Il dataset utilizzato è **DIV2K** [1], composto da 800 immagini di training e 100 immagini di validazione ad alta risoluzione ($\sim 2K$). Le immagini vengono scaricate automaticamente dal server ETH di Zurigo tramite il modulo `download_dataset.py` e salvate nella directory `data/raw/`.

Dalle immagini originali vengono generate le versioni degradate in modo controllato e riproducibile, salvate con una struttura organizzata sotto `data/degraded/`. Il sistema supporta la generazione automatica con naming conventions coerenti (ad esempio `data/degraded/gaussian/sigma_100/DIV2K_train_HR/`).

1.4 Pipeline generale

Il flusso di lavoro del progetto si articola in tre fasi principali:

1. **Degradazione:** Le immagini pulite vengono corrotte con degradazioni parametrizzabili (rumore gaussiano, dithering, salt-and-pepper). Ogni tipo di corruzione è implementato in un modulo dedicato e può essere applicato in batch all'intero dataset.
2. **Addestramento:** I modelli CNN vengono addestrati su coppie (immagine degradata, immagine pulita) estratte come patch casuali. Il sistema di training è unificato e supporta mixed precision (AMP), warmup del learning rate, scheduler coseno, early stopping e checkpointing automatico.
3. **Valutazione:** Le immagini restaurate vengono confrontate con i ground truth tramite metriche quantitative. L'inferenza su immagini full-resolution avviene tramite *sliding window* con blending pesato per evitare artefatti ai bordi delle patch.

Capitolo 2

Degradazioni

Le degradazioni sono il punto di partenza della pipeline: trasformano le immagini pulite in versioni corrotte che i modelli dovranno imparare a restaurare. Sono state implementate tre tipologie di degradazione, ciascuna parametrizzabile e riproducibile tramite seed.

2.1 Rumore Gaussiano

Il rumore gaussiano è la degradazione principale utilizzata negli esperimenti. Dato un pixel con valore x , l'immagine corrotta si ottiene come:

$$\tilde{x} = \text{clip}(x + \mathcal{N}(0, \sigma^2), 0, 255)$$

dove σ controlla l'intensità del rumore. Negli esperimenti è stato utilizzato $\sigma = 100$, un valore volutamente elevato per mettere alla prova la capacità di restauro dei modelli. Per garantire la riproducibilità, ogni immagine riceve un seed univoco calcolato come `seed + image_index`.

2.2 Quantizzazione e Dithering

La quantizzazione riduce la profondità di colore dell'immagine, mappando i 256 livelli originali per canale in un numero ridotto $L = 2^b$ dove b è il numero di bit per canale. Sono stati implementati quattro metodi:

2.2.1 Quantizzazione semplice

Applica una riduzione diretta della profondità:

$$q(x) = \text{round}\left(\frac{x}{255} \cdot (L - 1)\right) \cdot \frac{255}{L - 1}$$

2.2.2 Random Dithering

Prima della quantizzazione, aggiunge rumore uniforme nell'intervallo $[-\frac{s}{2}, \frac{s}{2}]$ dove $s = \frac{255}{L-1}$ è il passo di quantizzazione:

$$\tilde{x} = q\left(x + \mathcal{U}\left(-\frac{s}{2}, \frac{s}{2}\right) \cdot \text{noise_strength}\right)$$

Questo approccio spezza i pattern a bande tipici della quantizzazione semplice.

2.2.3 Ordered Dithering (Bayer)

Utilizza matrici di Bayer (2×2 , 4×4 o 8×8) come soglie spazialmente variabili. La matrice di Bayer viene normalizzata nell'intervallo $[-0.5, 0.5]$ e sommata all'immagine prima della quantizzazione, producendo pattern regolari e visivamente gradevoli.

2.2.4 Floyd-Steinberg Dithering

Algoritmo di *error diffusion* che processa l'immagine pixel per pixel. L'errore di quantizzazione $e = x - q(x)$ viene distribuito ai pixel adiacenti non ancora processati secondo il kernel:

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{bmatrix}$$

dove $*$ indica il pixel corrente. Questo produce risultati percettivamente superiori, al costo di un'elaborazione sequenziale.

2.3 Salt-and-Pepper

Il rumore *salt-and-pepper* sostituisce una frazione **amount** dei pixel dell'immagine con valori estremi: il parametro **salt_vs_pepper** controlla la proporzione tra pixel bianchi (255, *salt*) e neri (0, *pepper*).

2.4 Generazione automatica del dataset

Il modulo `generate_degraded_dataset.py` fornisce due funzioni principali:

- `generate_degraded_dataset`: applica una degradazione a tutte le immagini PNG/JPG di una directory;
- `generate_degraded_dataset_auto`: genera automaticamente la struttura di directory organizzata (es. `data/degraded/gaussian/sigma_100/DIV2K_train_HR/`), saltando la generazione se il dataset esiste già.

Capitolo 3

Infrastruttura comune

L'infrastruttura di supporto è condivisa da tutti i modelli e comprende il dataset PyTorch, il pipeline di training, il sistema di esperimenti, le loss function e le utilità di valutazione e inferenza.

3.1 Dataset e Data Loading

La classe `ImageEnhancementDataset` estende `torch.utils.data.Dataset` e gestisce il caricamento delle coppie (immagine degradata, immagine pulita) con estrazione di patch casuali.

3.1.1 Estrazione delle patch

Ogni immagine viene suddivisa in patch quadrate di dimensione configurabile (tipicamente 128×128). Per ogni immagine vengono estratte `patches_per_image` patch in posizioni casuali, così che la lunghezza effettiva del dataset è:

$$|\mathcal{D}| = N_{\text{immagini}} \times \text{patches_per_image}$$

Le immagini vengono mantenute in cache tramite `@lru_cache(maxsize=64)` per ridurre gli accessi a disco.

3.1.2 Data augmentation

Durante il training vengono applicate trasformazioni geometriche tramite la libreria `Albumentations`:

- `HorizontalFlip` (probabilità 0.5)
- `VerticalFlip` (probabilità 0.5)
- `RandomRotate90` (probabilità 0.5)

Le augmentation vengono applicate congiuntamente alla coppia degradata/pulita tramite il meccanismo `additional_targets` di `Albumentations`. La normalizzazione mappa i pixel da `[0, 255]` a `[-1, 1]` tramite $x' = (x/255 - 0.5)/0.5$. In validazione si applica solo la normalizzazione.

3.1.3 DataLoader

La funzione `get_dataloaders` crea i dataloader di training e validazione con le seguenti impostazioni:

- `pin_memory=True` per trasferimenti GPU asincroni
- `persistent_workers=True` per evitare il respawn dei worker
- `drop_last=True` nel training per batch di dimensione costante
- Worker init function che disabilita il threading OpenCV e imposta seed NumPy per worker

3.2 Funzioni di Loss

Sono state implementate diverse loss function, dalle più semplici a combinazioni più elaborate.

3.2.1 Combined Loss (L1 + SSIM)

La loss combinata è definita come:

$$\mathcal{L}_{\text{combined}} = \alpha \cdot \mathcal{L}_{\text{L1}} + \beta \cdot (1 - \text{SSIM})$$

dove $\alpha = 0.84$ e $\beta = 0.16$ sono i pesi di default. Per il calcolo dell'SSIM le immagini vengono convertite da $[-1, 1]$ a $[0, 1]$. Per stabilizzare il calcolo in mixed precision (FP16), vengono applicati un epsilon di clamping e una piccola iniezione di rumore gaussiano.

3.2.2 Perceptual Loss (VGG)

La `VGGPerceptualLoss` utilizza una rete VGG-16 pre-addestrata su ImageNet come estrattore di feature. Le feature vengono estratte a livelli configurabili (`relu2_2`, `relu3_3` di default) e la loss è calcolata come L1 pesata tra le feature map:

$$\mathcal{L}_{\text{perceptual}} = \sum_{l \in \text{layers}} w_l \cdot \|\phi_l(\hat{x}) - \phi_l(x)\|_1$$

dove ϕ_l indica le feature al livello l della VGG-16. I parametri della VGG sono congelati e gli input vengono normalizzati con le statistiche di ImageNet.

3.2.3 Combined Perceptual Loss

La `CombinedPerceptualLoss` unifica le tre componenti:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{L1}} + \beta \cdot (1 - \text{SSIM}) + \gamma \cdot \mathcal{L}_{\text{perceptual}}$$

Con i pesi tipici $\alpha = 0.6$, $\beta = 0.25$, $\gamma = 0.15$. Quando $\gamma = 0$ la rete VGG non viene caricata in memoria.

3.3 Sistema di Esperimenti

Il modulo `experiment.py` gestisce la creazione, il salvataggio e la ripresa degli esperimenti.

3.3.1 Struttura delle directory

Ogni esperimento viene salvato in una directory con timestamp:

```
experiments/{modello}/{degradazione}/{timestamp}_{nome}/
  checkpoints/      # Pesi del modello
  samples/          # Campioni visivi
  logs/             # Log TensorBoard
  config.json       # Configurazione completa
  history.json      # Metriche per epoch
```

3.3.2 Ripresa e fork

Il sistema supporta la ripresa automatica dell'ultimo esperimento. Se la configurazione è cambiata rispetto all'esperimento precedente, viene eseguito un *fork*: si crea un nuovo esperimento con i propri metadati, copiando il miglior checkpoint e la history dal padre. Questo consente di esplorare variazioni di iperparametri mantenendo la tracciabilità.

La funzione `compare_configs` confronta le configurazioni ignorando campi non significativi come `resume_from_checkpoint`, `num_epochs` e `device`.

3.4 Checkpointing

I checkpoint salvano lo stato completo dell'addestramento:

- Pesi del modello (`model_state_dict`)
- Stato dell'ottimizzatore e dello scheduler
- Epoch corrente e metriche
- Metadati opzionali (configurazione, ecc.)

Al caricamento, viene eseguita una validazione di integrità: tutti i tensori floating-point vengono controllati per la presenza di valori NaN o Inf, con errore esplicito che indica le chiavi corrotte.

In caso di errore OOM durante il training, viene salvato un checkpoint di emergenza. Se il file `best_model.pth` manca, il sistema lo ricrea a partire dal checkpoint di emergenza o periodico più recente.

3.5 Notifiche Telegram

Il modulo `telegram_notifier.py` invia notifiche sullo stato del training tramite l'API Telegram Bot. Vengono notificati: inizio training, aggiornamenti periodici sulle epoche, completamento, early stopping e errori OOM. Le credenziali vengono lette da un file `.env` (`TELEGRAM_BOT_TOKEN` e `TELEGRAM_CHAT_ID`).

Capitolo 4

Architetture dei Modelli

In questo capitolo vengono descritte le architetture dei modelli implementati. Tutti i modelli operano su immagini RGB normalizzate in $[-1, 1]$ e producono output nello stesso intervallo. Si distinguono due paradigmi: la **predizione diretta** dell'immagine pulita e il **residual learning**, dove il modello predice il rumore da sottrarre all'input.

4.1 UNet

Sviluppato da: Giuseppe Bellamacina

La UNet è un'architettura encoder-decoder con skip connection, originariamente proposta per la segmentazione biomedica e adattata qui al restauro di immagini. Il modello apprende una mappatura diretta dall'immagine degradata a quella pulita.

4.1.1 Architettura

L'encoder è composto da 4 stadi di downsampling, ciascuno formato da un blocco `DoubleConv` (due convoluzioni 3×3 con BatchNorm e ReLU) seguito da `MaxPool  $2 \times 2$` :

`inc(3  $\rightarrow$  64)  $\rightarrow$  down1(64  $\rightarrow$  128)  $\rightarrow$  down2(128  $\rightarrow$  256)  $\rightarrow$  down3(256  $\rightarrow$  512)  $\rightarrow$  down4(512  $\rightarrow$  1024)`

Il decoder ricostruisce la risoluzione spaziale tramite upsampling. Sono supportate due modalità:

- **Bilinear:** `nn.Upsample(scale_factor=2, mode='bilinear')` seguito da `DoubleConv` con canali dimezzati.
- **Transposed Convolution:** `ConvTranspose2d(kernel_size=2, stride=2)`.

Ad ogni livello del decoder, le feature dell'encoder vengono concatenate tramite skip connection dopo un eventuale padding per gestire discrepanze dimensionali. L'output finale è una convoluzione 1×1 che mappa da 64 a 3 canali, senza funzione di attivazione.

4.1.2 Configurazione

Parametro	Valore
Canali di ingresso/uscita	3 (RGB)
Feature iniziali	64
Modalità upsampling	Bilinear / Transposed Conv
Paradigma	Predizione diretta

Tabella 4.1: Parametri architetturali della UNet.

4.2 UNet Residual

Sviluppato da: Giuseppe Bellamacina

La UNet Residual condivide la stessa architettura encoder-decoder della UNet base, con una differenza fondamentale nel paradigma di apprendimento: il modello predice il **rumore** presente nell'immagine anziché l'immagine pulita. L'output denoised si ottiene per sottrazione:

$$\hat{x}_{\text{clean}} = x_{\text{noisy}} - f_{\theta}(x_{\text{noisy}})$$

dove f_{θ} è la rete che predice la componente di rumore.

4.2.1 Anti-checkerboard

Nella variante con transposed convolution, è stato applicato un fix per gli artefatti a scacchiera (*checkerboard artifacts*) tipici della deconvoluzione: il kernel della `ConvTranspose2d` è stato portato da 2×2 a 4×4 con `padding=1`, garantendo una copertura più uniforme dei pixel di output.

4.3 Attention UNet

Sviluppato da: Giuseppe Bellamacina

L'Attention UNet estende la UNet con meccanismi di *attention gate* (AG) nelle skip connection, permettendo al decoder di concentrarsi sulle regioni più rilevanti delle feature dell'encoder.

4.3.1 Attention Gate

L'attention gate calcola una mappa di attenzione spaziale a partire dal segnale di gating g (proveniente dal livello decoder inferiore) e dal segnale skip x (proveniente dall'encoder):

$$\alpha = \sigma(\psi(\text{ReLU}(W_g \cdot g + W_x \cdot x)))$$

dove W_g e W_x sono convoluzioni 1×1 con normalizzazione, ψ è una convoluzione 1×1 che riduce a un singolo canale, e σ è la funzione sigmoide. Le feature skip vengono modulate element-wise:

$$\hat{x} = \alpha \odot x$$

4.3.2 Livelli di attenzione

Il modello supporta due configurazioni:

- ("bottleneck",): attention gate applicato solo al primo livello del decoder (collegamento con il bottleneck).
- ("all",): attention gate applicato a tutti i livelli del decoder.

4.3.3 Normalizzazione

Il modello supporta sia `BatchNorm2d` che `GroupNorm` (con $\min(8, C)$ gruppi), selezionabile tramite il parametro `norm`.

4.3.4 Adaptive Blending

L'Attention UNet supporta un meccanismo opzionale di blending adattivo basato sull'intensità del rumore stimata. Se viene fornito un valore di `noise_sigma` durante l'inferenza:

$$\alpha = \min\left(\frac{\sigma_{\text{noise}}}{\sigma_{\text{trained}}}, 1.0\right), \quad \text{output} = \alpha \cdot x_{\text{input}} + (1 - \alpha) \cdot x_{\text{denoised}}$$

Questo permette di moderare l'intervento del modello quando il rumore è inferiore a quello su cui è stato addestrato.

4.4 DnCNN

Sviluppato da: Salvo Iurato

Il DnCNN (*Denoising Convolutional Neural Network*) è un'architettura puramente sequenziale che sfrutta il residual learning. A differenza delle architetture encoder-decoder, non utilizza alcuna forma di downsampling o skip connection.

4.4.1 Architettura

La rete è composta da 20 layer convoluzionali:

1. **Primo layer:** `Conv2d(3→64, 3×3)` con bias + ReLU (senza BatchNorm).
2. **Layer 2–19:** `Conv2d(64→64, 3×3)` senza bias + BatchNorm + ReLU.
3. **Ultimo layer:** `Conv2d(64→3, 3×3)` con bias (senza BatchNorm né attivazione).

Il rumore predetto viene sottratto dall'input:

$$\hat{x}_{\text{clean}} = x_{\text{noisy}} - f_{\theta}(x_{\text{noisy}})$$

In fase di inferenza, l'output viene clampato nell'intervallo $[-1, 1]$. I pesi sono inizializzati con Kaiming normal per le convoluzioni e costanti (1 / 0) per la BatchNorm.

4.5 Denoising Autoencoder

Sviluppato da: Salvo Iurato

Il Denoising Autoencoder è un'architettura encoder-decoder **senza skip connection**, progettata come baseline per il confronto con le architetture più avanzate.

4.5.1 Architettura

L'**encoder** esegue 4 stadi di downsampling tramite convoluzioni strided (3×3 , **stride=2**), ciascuno seguito da BatchNorm e LeakyReLU(0.2):

$$3 \xrightarrow{128 \rightarrow 64} 32 \xrightarrow{64 \rightarrow 32} 64 \xrightarrow{32 \rightarrow 16} 128 \xrightarrow{16 \rightarrow 8} 256$$

Il **decoder** ricostruisce la risoluzione tramite **Upsample(bilinear) + Conv2d(3×3)** per ogni stadio, con BatchNorm e LeakyReLU, fino a un layer finale con attivazione Tanh che produce l'output in $[-1, 1]$.

L'assenza di skip connection costringe il bottleneck a contenere tutta l'informazione necessaria per la ricostruzione, rendendo il modello più "lossy" ma anche più semplice.

4.6 RIDNet

Sviluppato da: Salvo Iurato

RIDNet (*Real Image Denoising Network*) è un modello che combina convoluzioni dilatate e attention a livello di canale per catturare informazioni multi-scala senza ricorrere a operazioni di pooling.

4.6.1 Enhancement Attention Block (EAB)

L'unità fondamentale di RIDNet è l'*Enhancement Attention Block*, composto da:

1. Tre convoluzioni con dilatazione crescente ($d = 1, 2, 3$), ciascuna seguita da BatchNorm e ReLU. Questo espande il campo recettivo senza ridurre la risoluzione spaziale.
2. Un modulo **ChannelAttention** di tipo Squeeze-and-Excitation: Global Average Pooling $\rightarrow$ Conv 1×1 ($C \rightarrow C/r$) $\rightarrow$ ReLU $\rightarrow$ Conv 1×1 ($C/r \rightarrow C$) $\rightarrow$ Sigmoid, con fattore di riduzione $r = 16$.
3. Una connessione residuale locale con fattore di scala:

$$\text{EAB}(x) = x + s \cdot \text{CA}(\text{dilated_branch}(x))$$

dove s è il **residual_scale** (default 1.0, negli esperimenti 0.1).

4.6.2 Architettura completa

1. **Estrazione feature:** Conv2d($3 \rightarrow 64$, 3×3) + ReLU.
2. **Enhancement:** Sequenza di N EAB (tipicamente $N = 8$).
3. **Ricostruzione:** Conv2d($64 \rightarrow 3$, 3×3) che predice il rumore.
4. **Residual globale:** $\hat{x} = x - f_{\theta}(x)$.

In inferenza l'output viene clampato a $[-1, 1]$.

4.7 Pix2Pix

Sviluppato da: Daniele Barbagallo

Pix2Pix è un framework di *image-to-image translation* basato su Generative Adversarial Network (GAN) condizionale. Il sistema è composto da un **generatore** e un **discriminatore** addestrati in modo avversariale.

4.7.1 Generatore

Il generatore ha un'architettura UNet-like con 7 livelli di downsampling, progettata per input 256×256 :

Encoder (6 layer + bottleneck):

$$\text{enc}_1(3 \rightarrow 64) \rightarrow \text{enc}_2(64 \rightarrow 128) \rightarrow \dots \rightarrow \text{enc}_6(512 \rightarrow 512) \rightarrow \text{bottleneck}(512 \rightarrow 512)$$

Ogni layer usa `Conv2d(4×4, stride=2) + BatchNorm + LeakyReLU(0.2)`. Il primo layer non ha BatchNorm.

Decoder (6 layer + finale): `ConvTranspose2d(4×4, stride=2) + BatchNorm + ReLU`, con Dropout(0.5) nei primi 3 layer. Skip connection via concatenazione con il corrispondente encoder. Il layer finale produce 3 canali con attivazione Tanh (o senza, per residual learning).

4.7.2 PatchGAN Discriminator

Il discriminatore opera su *patch* locali dell'immagine, classificando regioni 70×70 come reali o generate. L'input è la concatenazione dell'immagine condizionale (degradata) e dell'immagine target/generata (6 canali totali).

L'architettura consiste in 4 blocchi convoluzionali (4×4 , stride progressivamente ridotto) che riducono le feature da 64 a 512 canali, seguiti da una convoluzione finale 4×4 che produce un output a singolo canale (logits, senza sigmoide).

4.7.3 Modalità di apprendimento

Nell'esperimento condotto, il generatore opera in **residual learning**: predice $\hat{n} = G(x_{\text{noisy}})$ (il rumore stimato) e l'immagine pulita si ottiene come:

$$\hat{x} = \text{clamp}(x_{\text{noisy}} - \hat{n}, -1, 1)$$

Il rumore predetto viene inoltre clampato a ± 0.6 per stabilità.

4.8 NAFNet

Sviluppato da: Mattia Campanella

Sezione da completare — codice non ancora presente sul branch main.

4.9 GAN

Sviluppato da: Daniele Barbagallo

Sezione da completare — codice non ancora presente sul branch main.

4.10 Riepilogo delle architetture

Modello	Tipo	Residual	Attention	Attivazione output
UNet	Enc-Dec + skip	No	No	Nessuna
UNet Residual	Enc-Dec + skip	Sì	No	Nessuna
Attention UNet	Enc-Dec + skip	No	AG (bottleneck/all)	Nessuna
DnCNN	Sequenziale (20 layer)	Sì	No	Clamp $[-1, 1]$
Denoising AE	Enc-Dec (no skip)	No	No	Tanh
RIDNet	Feature + EAB	Sì	Channel (SE)	Clamp $[-1, 1]$
Pix2Pix Gen.	UNet-like (7 livelli)	Configurabile	No	Tanh / Nessuna

Tabella 4.2: Confronto delle architetture implementate.

Capitolo 5

Training

5.1 Pipeline di addestramento

Il training loop è implementato nel modulo `trainer.py` e supporta tutte le architetture tramite un'interfaccia unificata. La funzione `run_training` orchestra l'intero processo: warmup del learning rate, alternanza train/validation, checkpointing, early stopping e logging.

5.1.1 Mixed Precision (AMP)

Tutti gli esperimenti (eccetto il Denoising Autoencoder) utilizzano Automatic Mixed Precision di PyTorch: le operazioni nel forward pass vengono eseguite in FP16 con `torch.autocast`, mentre la loss viene sempre calcolata in FP32 per stabilità numerica. Un `GradScaler` gestisce il ridimensionamento dei gradienti.

La validazione viene eseguita **sempre in full precision**, indipendentemente dall'impostazione AMP, per evitare instabilità causate dall'interazione tra BatchNorm in modalità eval e FP16.

5.1.2 Learning Rate Warmup

I primi `warmup_epochs` (tipicamente 5–8) utilizzano un warmup lineare del learning rate da `warmup_start_lr` (o $lr \times \frac{\text{epoch}}{\text{warmup_epochs}}$ se non specificato) fino al learning rate target. Lo scheduler principale (CosineAnnealing o StepLR) viene attivato solo al termine del warmup.

5.1.3 Gradient Clipping

I gradienti vengono clippati tramite `clip_grad_norm_` con norma massima configurabile (tipicamente 0.5–1.0). In caso di AMP, i gradienti vengono prima “unscaled” dal `GradScaler` prima del clipping.

5.1.4 Early Stopping

Il contatore di patience viene incrementato solo nelle epoche di validazione in cui la val loss non migliora. Questo permette di configurare la validazione a intervalli (es. ogni 2 epoche) senza penalizzare il conteggio.

5.1.5 Gestione errori OOM

In caso di errore *Out of Memory* durante il training, il sistema:

1. Salva un checkpoint di emergenza
2. Libera la memoria CUDA
3. Stampa un messaggio diagnostico con suggerimenti (ridurre batch/patch size, abilitare AMP)

5.2 Training Pix2Pix

Il training del Pix2Pix segue il paradigma GAN con addestramento alternato di generatore e discriminatore.

5.2.1 Training del Discriminatore

Il discriminatore riceve coppie di immagini (condizionale + target):

- **Coppie reali:** $(x_{\text{degraded}}, x_{\text{clean}}) \rightarrow \text{label } 1$
- **Coppie false:** $(x_{\text{degraded}}, G(x_{\text{degraded}})) \rightarrow \text{label } 0$

La loss del discriminatore è:

$$\mathcal{L}_D = \frac{1}{2} (\text{BCE}(D(x, y_{\text{real}}), 1) + \text{BCE}(D(x, y_{\text{fake}}), 0))$$

5.2.2 Training del Generatore

La loss del generatore combina la componente avversariale con una loss di ricostruzione:

$$\mathcal{L}_G = \text{BCE}(D(x, G(x)), 1) + \lambda_{\text{L1}} \cdot \mathcal{L}_{\text{reconstruction}}(G(x), y)$$

dove $\lambda_{\text{L1}} = 150$ e la loss di ricostruzione è la **CombinedLoss** (L1 + SSIM). Il fattore λ_{L1} elevato bilancia la componente avversariale, che tende a dominare nelle prime fasi del training.

5.2.3 Diagnostica del rumore

In modalità residual learning, vengono monitorate statistiche sul rumore predetto: media, deviazione standard, valore massimo, percentuale di outlier (valori che superano il clamp prima del clamping) e percentuale di pixel clippati nell'output.

5.3 Configurazioni degli esperimenti

La tabella seguente riassume gli iperparametri principali di ciascun esperimento condotto. Tutti gli esperimenti utilizzano il dataset DIV2K con rumore gaussiano $\sigma = 100$, salvo diversa indicazione.

Modello	Loss	LR	Epoche	Batch	Patch	Warmup	Patience
UNet (bilinear)	L1+SSIM	1e-4	36	16	128	5	5
UNet Res. (bilinear)	L1+SSIM+VGG	1e-4	36	16	128	5	5
UNet Res. (transconv)	L1+SSIM+VGG	1e-4	36	16	128	5	5
Att. UNet (bilinear, all)	L1+SSIM+VGG	4e-5	36	16	128	8	5
Att. UNet (bilinear, btlmk)	L1+SSIM+VGG	4e-5	36	16	128	8	3
Att. UNet (transconv, all)	L1+SSIM+VGG	4e-5	36	16	128	8	5
Att. UNet (transconv, btlmk)	L1+SSIM+VGG	4e-5	36	16	128	8	5
DnCNN-20	L1+SSIM	1e-4	100	64	80	—	10
Denoising AE	L1+SSIM	1e-4	200	40	128	2	15
RIDNet (8 EAB)	L1+SSIM	1e-4	150	8	256	5	10
Pix2Pix	GAN+L1+SSIM	G:2e-4	150	8	384	—	25

Tabella 5.1: Iperparametri di training per ciascun esperimento. Le loss L1+SSIM usano pesi $\alpha=0.84, \beta=0.16$; le loss L1+SSIM+VGG usano $\alpha=0.6, \beta=0.25, \gamma=0.15$ (salvo varianti bottleneck con 0.65/0.3/0.05).

5.3.1 Note sugli esperimenti

- **Ottimizzatore:** Tutti i modelli usano AdamW (`weight_decay` = 10^{-5}), eccetto DnCNN e RIDNet che usano Adam (`weight_decay` = 10^{-4}) e Pix2Pix che usa Adam con $\beta_1 = 0.5$.
- **Scheduler:** CosineAnnealingLR per la maggior parte dei modelli; StepLR (`step=1`, $\gamma = 0.96$) per DnCNN; ReduceLROnPlateau per il discriminatore Pix2Pix; nessuno scheduler per il generatore Pix2Pix.
- **Attention UNet — varianti bottleneck:** Utilizzano GroupNorm invece di BatchNorm, layer percettuali meno profondi (`relu1_2` + `relu2_2` anziché `relu2_2` + `relu3_3`) e peso percettuale ridotto ($\gamma = 0.05$).
- **RIDNet:** Utilizza 8 EAB con `residual_scale` = 0.1, feature a 96 canali (anziché 64) e gradient clip a 0.5 (più restrittivo della norma 1.0 usata dagli altri).
- **Dithering:** Sono stati condotti due esperimenti anche con degradazione di tipo quantizzazione a 2 bit con random dithering, usando la UNet base.

Capitolo 6

Valutazione

6.1 Metriche

Le prestazioni dei modelli vengono valutate tramite quattro metriche quantitative, calcolate su immagini normalizzate in $[0, 1]$.

6.1.1 PSNR (Peak Signal-to-Noise Ratio)

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{\text{MAX}}{\sqrt{\text{MSE}}} \right) \quad [\text{dB}]$$

dove $\text{MAX} = 1.0$ è il valore massimo del segnale. Valori più alti indicano maggiore fedeltà al ground truth.

6.1.2 SSIM (Structural Similarity Index)

L'SSIM è calcolato con una finestra di dimensione 11×11 tramite `F.avg_pool2d`:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

con $C_1 = (0.01)^2$ e $C_2 = (0.03)^2$. Valori più vicini a 1 indicano maggiore similarità strutturale.

6.1.3 MAE e MSE

Rispettivamente la media del valore assoluto e del quadrato delle differenze pixel-wise, calcolate tramite `F.l1_loss` e `F.mse_loss`.

6.2 Inferenza su immagini full-resolution

Le immagini del dataset DIV2K hanno risoluzione $\sim 2000 \times 2000$ pixel, troppo grande per essere processate in un singolo forward pass. L'inferenza avviene quindi tramite **sliding window** con blending pesato.

6.2.1 Sliding Window

L'immagine viene suddivisa in patch sovrapposte di dimensione `patch_size` (tipicamente 256) con `overlap` (tipicamente 32–64) pixel di sovrapposizione:

$$\text{stride} = \text{patch_size} - \text{overlap}$$

6.2.2 Hann Window Blending

Per evitare artefatti ai bordi delle patch, ogni output viene pesato con una finestra 2D di Hann, ottenuta come prodotto esterno di due finestre 1D:

$$w(i, j) = w_H(i) \cdot w_H(j), \quad w_H(k) = 0.5 - 0.5 \cos\left(\frac{2\pi k}{N-1}\right)$$

L'output finale per ogni pixel è la media pesata dei contributi di tutte le patch che lo contengono:

$$I_{\text{out}}(i, j) = \frac{\sum_p w_p \cdot O_p(i, j)}{\sum_p w_p}$$

Questo produce transizioni graduali e naturali tra patch adiacenti.

6.3 Valutazione del dataset

La classe `ImageRestorationEvaluator` orchestra la valutazione sull'intero dataset:

1. Per ogni coppia (degradata, pulita): inferenza con sliding window, calcolo delle metriche.
2. Aggregazione: media e deviazione standard su tutte le immagini.
3. Report: salvataggio in formato JSON con metriche per-immagine e aggregate, stampa delle immagini migliori e peggiori.

Bibliografia

- [1] E. Agustsson and R. Timofte, “NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study,” in *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017.